University of Piraeus
Department of Digital Systems

Mobile and Wireless Communications Security

ΟΝΟΜΑΤΕΠΩΝΥΜΟ	ΑΡΙΘΜΟΣ ΜΗΤΡΩΟΥ	EMAIL	ΗΜΕΡΟΜΗΝΙΑ
ELVIS AVDULI	E22001	avdulielvis@gmail.com	30/04/2026

8th Semester

Marketplace DApp Report

This project implements a simple decentralized marketplace application (DApp) using Solidity, Truffle, Ganache, MetaMask, HTML, CSS, and JavaScript with Web3.js. The goal of the project is to understand how data can be stored and managed on the blockchain and how a web interface can interact with a smart contract in a practical scenario. The DApp allows users to list products for sale and purchase products from other users through Ethereum transactions.

The smart contract is implemented in Marketplace.sol and represents the core business logic of the application. A Product struct is used to store each product's information, including a unique id, product name, price in wei, seller address, and sale status. Products are stored in a mapping from id to Product, while an auto-increment counter called productCount is used to assign unique ids to newly listed products. This design provides simple and direct access to product records and supports predictable retrieval from the front-end.

The contract provides a listProduct function that allows any user to create a new listing. Before storing the listing, the contract verifies that the product name is not empty and that the price is greater than zero. After successful validation, the product is saved in storage and a ProductListed event is emitted. The contract also provides a purchaseProduct function that allows a user to buy an existing product by sending Ether. The function checks that the product exists, has not already been sold, and that the payment amount exactly matches the product price. It also blocks the seller from buying their own product. If all checks pass, the product is marked as sold and the payment is transferred directly to the seller.

The front-end is implemented with index.html, styles.css, and app.js. Web3.js is loaded through a CDN and is used to connect the browser application to MetaMask and the deployed smart contract. When the user clicks the connect button, the application requests account access from MetaMask and stores the selected wallet address. The interface then loads the contract ABI and deployed address from the Truffle artifact file and creates a contract instance. After this setup, the application can read product data from the blockchain and submit signed transactions through MetaMask.

The user interface supports the two main marketplace actions required by the assignment. First, it provides a form where users enter a product name and price in ETH. In JavaScript, the price is converted from ETH to wei before calling listProduct, ensuring correct on-chain value format. Second, it displays all unsold products and

adds a buy button for each one. Pressing the buy button sends a purchaseProduct transaction with the exact product price as transaction value. After each successful transaction, the product list is refreshed so the interface always reflects the latest on-chain state.

For local blockchain development and testing, Ganache is used as the Ethereum network and Truffle is used for compiling and deployment. The project is configured with a development network in truffle-config.js and the contract is deployed using the migration script 1_deploy_marketplace.js. During implementation, a deployment issue appeared due to EVM compatibility between compiler output and local Ganache behavior. This was resolved by setting the Solidity compiler evmVersion to paris in the Truffle configuration. After this change, deployment and tests completed successfully.

Several practical issues were handled during integration. The first issue was that opening the front-end directly with a file URL prevented loading of JSON artifacts through fetch, so a local HTTP server was required to serve the project files. The second issue was network mismatch between MetaMask and the deployed contract network, which was solved by ensuring MetaMask uses the same Ganache chain where the contract was deployed. The third issue involved transaction reverts when payment did not match the exact price, which was solved by strict ETH to wei conversion in the front-end and sending the precise value expected by the contract.

To execute the Marketplace DApp, first start a local Ethereum blockchain using Ganache on port 7545, then open the project folder in PowerShell and run Truffle compile followed by Truffle migrate --reset --network development so the smart contract is deployed to the local chain and artifact data is updated; next, run a local HTTP server from the same project directory (for example with npx http-server . -p 5500 -c-1) and open the served URL in your browser instead of opening the HTML file directly, because the app needs HTTP access to load contract artifacts. After the page loads, configure MetaMask to the same Ganache network (matching RPC URL, chain ID, and account set), click Connect MetaMask in the UI, and then test the two workflows by listing a product (price entered in ETH and converted to wei in the app) and purchasing an unsold product with the exact required value; if Ganache is restarted, redeploy the contract again and refresh the browser so the frontend points to a valid on-chain contract instance.